STARVNT CORE PLATFORM

MASTER EXECUTION MISSION

EVENT CORE AUTHORITY -> EVENT COMMERCE OPERATING FLOW

Production Event Lifecycle + Real Commercial Transaction Infrastructure

DAY 6
TRUST THE EVENT CORE

DAY 7
TRUST THE COMMERCIAL FLOW

MISSION STANDARD One StarVnt. One connected operating model. Working proof over documentation.

OWNER **Keshab - Technical Lead / Core Platform**

CONFIDENTIAL - INTERNAL ENGINEERING EXECUTION DOCUMENT

00 MASTER MISSION

Day 6 and Day 7 are one continuous architectural progression, not two unrelated feature-development days. Day 6 establishes operational authority in the Event Core. Day 7 proves that the same Core can support a real commercial transaction without breaking domain ownership.

```
IDENTITY -> ORGANIZATION -> MEMBERSHIP -> RBAC
    -> EVENT CORE -> LIFECYCLE -> REQUIREMENTS -> READINESS
    -> VENDOR / MATCHING -> QUOTE -> CUSTOMER APPROVAL -> BOOKING
    -> PAYMENT INTENT -> RAZORPAY ORDER -> CHECKOUT
    -> SERVER VERIFICATION -> PAYMENT TRANSACTION -> PAYMENT CONFIRMED
    -> OUTBOX -> WORKER -> BOOKING CONFIRMATION
    -> EVENT DEPENDENCY -> READINESS RECALCULATION
    -> AUTOMATION -> CUSTOMER / AURA+
```

DAY 6 Make the Event Core trustworthy enough to become the operational heart of StarVnt.

DAY 7 Make the commercial transaction flow trustworthy enough to turn an approved booking into a verified business transaction.

01 ABSOLUTE ARCHITECTURAL PRINCIPLE

StarVnt is one connected Event Operating & Execution Platform. Before writing new code, verify what already exists, identify the correct domain owner, inspect data models and contracts, and extend rather than duplicate.

```
REVIEW -> UNDERSTAND -> TRACE EXISTING IMPLEMENTATION -> IDENTIFY DOMAIN OWNER
-> CHECK FOR DUPLICATION -> REUSE -> EXTEND -> INTEGRATE -> TEST -> PROVE
```

GOLDEN RULE NEW IDEA != NEW SYSTEM

02 DAY 6 + DAY 7 STRATEGIC PROGRESSION

Day 6 produces a trustworthy Event Core. Day 7 connects that authority to commercial execution. The boundary between the two must remain explicit.

DAY 6	DAY 7
Event Core authority	Commercial transaction authority

DAY 6	DAY 7
Lifecycle state machine	Payment / booking execution
Requirements + readiness	Quote -> approval -> booking
Business events + outbox	Payment events + webhook
Automation + audit	Verification + reconciliation
Failure recovery	Commercial failure recovery
E2E Event proof	E2E money-moving proof

03 WHAT THIS MISSION IS NOT

- Not a CRUD exercise or an Event UI exercise.
- Not a standalone Razorpay integration.
- Not permission to build a second Event, Payment, Booking, Automation or Audit system.
- Not permission to implement every future Event OS or Payment Infrastructure feature.
- Not a documentation-only assignment or a mock payment demo.

STANDARD Working proof > documentation

04 EXISTING STARVNT DOMAIN OWNERSHIP

Domain	Authority
Identity	Authentication / identity
Organization	Tenant / membership
RBAC	Roles / permissions
Revenue	Leads / pipeline / sales context
Customer	Customer identity / context
Event Core	Event lifecycle / event-level authority

Domain	Authority
Applicability	Required / Optional / Not Applicable / Customer decision
Requirement	Event requirements
Vendor	Vendor business data
Matching	Vendor suitability
Quote	Commercial quotation
Booking	Booking lifecycle
Payment	Payment transaction / financial lifecycle
QR / Entry	Entry operations
Automation	Trigger -> condition -> action
Aura+	Intelligence / recommendation / prediction
Realtime	Realtime propagation
Audit	Authoritative activity history

OWNERSHIP RULE No domain silently absorbs another domain's authority.

05 EVENT CORE - DAY 6

Event Core is the operational heart of StarVnt. It owns lifecycle, event-level state and operational authority, while consuming information from other domains through explicit contracts.

EVENT CORE OWNS:
Lifecycle | Event state | Event context | Event dependencies | Readiness
Commands | State changes | Event history | Event-level permissioning | Lifecycle events

It does not become the owner of vendor profiles, payment ledgers, booking internals, guest records, financial settlement, Aura+ intelligence or realtime infrastructure.

06 EVENT LIFECYCLE

```
DRAFT -> MANIFESTING -> CONFIRMED -> ONGOING -> COMPLETED
```

Allowed transition	Expected
DRAFT -> MANIFESTING	ALLOW
MANIFESTING -> CONFIRMED	ALLOW
CONFIRMED -> ONGOING	ALLOW
ONGOING -> COMPLETED	ALLOW

Invalid transition	Expected behaviour
DRAFT -> COMPLETED	Reject; no state change
COMPLETED -> ONGOING	Reject; no state change
ONGOING -> DRAFT	Reject; no state change
CONFIRMED -> MANIFESTING	Reject; no state change

07 UNIVERSAL STATE TRANSITION CONTRACT

```
COMMAND
  -> AUTHORIZATION
  -> VALIDATION
  -> STATE TRANSITION
  -> DOMAIN EVENT
  -> TRANSACTIONAL OUTBOX
  -> WORKER / CONSUMER
  -> AUTOMATION / INTEGRATION
  -> DOMAIN ACTION
  -> VERIFICATION
  -> AUDIT
```

RULE A lifecycle transition is not a simple database status update.

08 EVENT CREATION

```
AUTHORIZED USER -> CREATE EVENT COMMAND -> ORGANIZATION CONTEXT -> VALIDATION
-> EVENT CREATED -> STATE = DRAFT -> EVENT_CREATED -> OUTBOX -> WORKER -> CONSUMERS
```

Event creation must reuse existing Identity, Organization, Membership and RBAC foundations. Event records are organization-scoped.

09 EVENT PERMISSIONING

```
USER -> ORGANIZATION -> MEMBERSHIP -> ROLE -> PERMISSION -> EVENT ACCESS -> COMMAND
```

Action	Permission
Create Event	Event Create
View Event	Event Read
Edit Draft	Event Update
Manage Event	Event Manage
Confirm Event	Event Confirm
Operate Event	Event Operate
Complete Event	Event Complete

Use equivalent permission names already present in the codebase when they provide the same capability. Do not duplicate authorization logic.

10 TENANT ISOLATION

```
ORG-A -> EVENT-A
ORG-B -> EVENT-B
```

```
ORG-A USER -> ORG-B EVENT -> DENIED
```

- Test Event ID manipulation.
- Test Organization ID manipulation.
- Test wrong JWT, organization, role and permission.
- Test direct API access and cross-tenant lookup/mutation.

EXPECTED DENIED + NO DATA LEAK + NO STATE CHANGE + NO FALSE EVENT

11 CHECKLIST + REQUIREMENT INTEGRATION

```
EVENT CREATED -> CHECKLIST / REQUIREMENTS -> DEPENDENCIES -> PROGRESS -> READINESS
```

Reuse the existing Checklist and Requirement capabilities. Checklist progress must contribute to actual Event readiness; it must not become an arbitrary percentage display.

12 APPLICABILITY + REQUIREMENT PRINCIPLE

```
EVENT CONTEXT -> APPLICABILITY -> REQUIREMENT -> DEPENDENCY -> READINESS

REQUIRED | OPTIONAL | NOT_APPLICABLE | CUSTOMER_REQUESTED | CUSTOMER_DECLINED
```

Avoid hardcoding separate workflows for every event type. Use the shared Event Context, Applicability and Requirement model.

13 EVENT READINESS

```
REQUIREMENT -> DEPENDENCY -> CURRENT STATUS -> READINESS IMPACT
```

Example dependency	Status
Venue	READY
Decorator	READY

Example dependency	Status
Catering	PAYMENT PENDING
Guest Count	INCOMPLETE

READINESS RULE Readiness must be based on actual dependencies, not an arbitrary progress number.

14 MANIFESTING -> CONFIRMED

```
CONFIRM EVENT -> AUTHORIZATION -> VALID EVENT? -> REQUIRED CONTEXT?
-> REQUIRED REQUIREMENTS? -> CRITICAL DEPENDENCIES? -> READY? -> CONFIRMED
```

```
VALIDATION FAILED -> EVENT REMAINS MANIFESTING -> NO EVENT_CONFIRMED
```

Customer-facing blockers should explain the business reason, e.g. "Catering confirmation is waiting for the required advance payment."

15 BUSINESS EVENTS + CANONICAL EVENT CONTRACT

Event	Purpose
EVENT_CREATED	Event creation occurrence
EVENT_STATE_CHANGED	Lifecycle mutation signal
EVENT_CONFIRMED	Confirmed Event signal
EVENT_STARTED	Operational start signal
EVENT_COMPLETED	Completion signal
CHECKLIST_UPDATED	Checklist/requirement change
READINESS_CHANGED	Readiness recalculation signal

Envelope concepts should include eventId, eventType, eventVersion, organizationId, entityType, entityId, occurredAt, source and payload. Use existing naming and infrastructure where available.

16 TRANSACTIONAL OUTBOX

```
DATABASE STATE CHANGE + OUTBOX RECORD -> COMMIT -> WORKER -> CONSUMERS
```

OBJECTIVE Prevent "database updated but event silently lost". Reuse the existing reliability foundation.

17 AUTOMATION INTEGRATION

```
EVENT_CREATED -> AUTOMATION -> RULE -> ACTION
EVENT_CONFIRMED -> AUTOMATION -> PRE-EVENT WORKFLOW
EVENT_COMPLETED -> AUTOMATION -> POST-EVENT WORKFLOW
```

Do not create an Event-specific automation engine. Reuse the central Automation Engine.

18 AURA+ BOUNDARY

```
EVENT CORE -> AUTHORIZED EVENT CONTEXT -> AURA+ -> ANALYSIS -> SIGNAL -> RECOMMENDATION

AURA+ RECOMMENDATION -> AUTHORIZED HANDOFF -> CORE / CORRECT DOMAIN
-> AUTOMATION / COMMAND -> ACTION -> RESULT -> AURA+ CONTEXT
```

PROHIBITED Aura+ must never directly mutate authoritative Event Core, Booking or Payment state.

19 VENDOR / BOOKING / PAYMENT / QR / REALTIME BOUNDARIES

```
EVENT CORE -> EVENT CONTEXT -> VENDOR MATCHING -> QUOTE -> BOOKING

EVENT -> REQUIREMENT -> BOOKING -> PAYMENT

EVENT -> EVENT ACCESS CONTEXT -> QR / ENTRY -> VALIDATION -> ENTRY RESULT
```

```
EVENT CORE -> CANONICAL EVENT -> REALTIME HUB -> CONNECTED CLIENTS
```

Event Core may consume relationships, status and dependency information from these domains, but must not become their canonical owner.

20 FUTURE EVENT OS COMPATIBILITY

Advanced future services such as Guest, Tracking, Zone, Heatmap, Event Manager AI, FTAura, Announcer, Navigation, Networking, Feed, Album, Memory Ticket, Gift AI, Prediction and Realtime should not be implemented as a mass build during Day 6/7. Create safe contracts they can later consume.

```
EVENT CORE -> CANONICAL EVENTS -> DEFINED CONTRACTS -> FUTURE SERVICES
```

21 AUDIT + IDEMPOTENCY + FAILURE RECOVERY

Important lifecycle actions must be auditable with actor, organization, Event, action, previous state, new state, source and timestamp. Automation and Aura+ handoffs should preserve their triggering event or Recommendation ID.

```
CONFIRM EVENT x100 -> ONE EFFECTIVE STATE TRANSITION
                  -> ONE EFFECTIVE BUSINESS EVENT
                  -> ONE EFFECTIVE ACTION
```

```
EVENT_CONFIRMED -> OUTBOX -> WORKER FAILURE -> RETRY -> RECOVERY
                          -> IDEMPOTENT PROCESSING
                          -> ONE EFFECTIVE BUSINESS OUTCOME
```

RELIABILITY TARGET At-least-once delivery + idempotent processing + exactly-once effective business outcome. Do not claim literal exactly-once message delivery.

22 EVENT HEALTH / DIGITAL TWIN / EVENT MEMORY COMPATIBILITY

Expose reliable Event context for future Event Health: readiness, critical dependencies, vendor risk, payment readiness, timeline, guest readiness and operational risk. Do not build a massive Health AI now.

Keep Event Core as the canonical Event lifecycle source for a future Digital Twin. Other domains contribute authoritative information through contracts; no second Event representation should be created.

Preserve lifecycle, commercial and audit history so future Event Memory can derive planned vs actual, budget vs actual, vendor performance, delays, issues, guest count, requirement history, cost deviations, decisions, payments and bookings.

23 DAY 6 END-TO-END PROOF

```
LOGIN -> ORGANIZATION -> ROLE -> CREATE EVENT -> DRAFT -> CONFIGURE
-> REQUIREMENTS / CHECKLIST -> READINESS -> MANIFESTING -> CONFIRM
-> AUTHORIZATION -> VALIDATION -> CONFIRMED -> EVENT_CONFIRMED
-> OUTBOX -> WORKER -> AUTOMATION / CONSUMERS -> AUDIT
-> START EVENT -> ONGOING -> EVENT_STARTED
-> COMPLETE EVENT -> COMPLETED -> EVENT_COMPLETED -> AUDIT / EVENT HISTORY
```

EVIDENCE This must be real working API/database/event/outbox/worker/audit behaviour, not documentation only.

24 DAY 7 - EVENT COMMERCE OPERATING FLOW

Day 7 begins where Day 6 establishes operational authority. The commercial chain connects existing Revenue, Requirement, Vendor, Matching, Quote and Booking capabilities to a production-grade Payment foundation.

```
LEAD -> QUALIFY -> REQUIREMENT -> MATCH -> QUOTE -> CUSTOMER APPROVAL
-> BOOKING -> PAYMENT INTENT -> RAZORPAY ORDER -> CHECKOUT
-> SERVER VERIFICATION -> PAYMENT TRANSACTION -> PAYMENT CONFIRMED
-> OUTBOX -> WORKER -> BOOKING CONFIRMATION
-> EVENT DEPENDENCY -> READINESS -> AUTOMATION -> CUSTOMER / AURA+
```

25 EXISTING REVENUE / VENDOR / BOOKING INTEGRATION

If Lead, Qualification, Matching, Quote or Booking capabilities already exist, Keshab must integrate with them rather than recreate them. The role is to connect existing domain capabilities into one commercial flow.

INTEGRATION RULE Do not rebuild Lead CRM, qualification, vendor matching, quote or booking merely to make the Day 7 demo self-contained.

26 PAYMENT FOUNDATION

Day 7 must establish a proper Payment domain foundation. Payment Intent, Payment Transaction, Gateway Order and Booking are distinct concepts with clean boundaries.

Payment object	Meaning / required context
Payment Intent	A defined commercial obligation requiring payment; supports purpose, amount due, currency and references.
Payment Transaction	An actual payment attempt / financial transaction with gateway and failure metadata.
Gateway Order	The gateway-side order created from the server-controlled payable amount.
Booking	The commercial booking whose lifecycle remains Booking-owned.

27 PAYMENT INTENT

A Payment Intent should support concepts such as paymentIntentId, organizationId, eventId, bookingId, quoteId, purpose, amountDue, currency, status, createdAt and updatedAt.

```
BOOKING_ADVANCE
BOOKING_BALANCE
```

Do not permanently design Payment around only one payment purpose. The data model must remain extensible.

28 PAYMENT TRANSACTION

A Payment Transaction should support concepts such as transactionId, paymentIntentId, bookingId, eventId, gateway, gatewayOrderId, gatewayPaymentId, status, amount, currency, capturedAmount, failureCode, failureReason, attempt metadata and timestamps.

```
PAYMENT INTENT -> PAYMENT TRANSACTION / ATTEMPT -> GATEWAY
```

CRITICAL Do not collapse Quote, Booking, Payment Intent, Transaction, Refund and Settlement into one generic record.

29 SERVER-CONTROLLED AMOUNT

```
QUOTE / BOOKING -> AUTHORITATIVE COMMERCIAL AMOUNT -> PAYMENT INTENT
-> RAZORPAY ORDER -> CHECKOUT
```

SECURITY RULE The client must never be the authority for the payable amount.

30 REAL RAZORPAY TEST MODE

```
STARVNT BOOKING -> PAYMENT INTENT -> REAL RAZORPAY TEST ORDER
-> REAL CHECKOUT -> REAL TEST PAYMENT -> SERVER VERIFICATION -> WEBHOOK
```

A mocked payment-success response cannot be the primary Day 7 proof.

31 CHECKOUT != PAYMENT CONFIRMATION

Frontend checkout success is only a gateway interaction result. StarVnt payment confirmation must be a server-controlled business outcome based on verified gateway information.

```
CHECKOUT RESULT -> SERVER -> SIGNATURE VERIFICATION -> WEBHOOK VERIFICATION
-> AMOUNT RECONCILIATION -> CURRENCY RECONCILIATION
-> PAYMENT INTENT MATCH -> TRANSACTION UPDATE -> PAYMENT_CONFIRMED
```

32 SERVER-SIDE VERIFICATION

Verify the relevant checkout signature server-side using the appropriate secret. Verify webhook authenticity using the configured webhook secret and the raw request body where the gateway signature is defined over the raw payload.

```
EXPECTED AMOUNT = PAYMENT INTENT AMOUNT = VERIFIED GATEWAY AMOUNT
EXPECTED CURRENCY = VERIFIED TRANSACTION CURRENCY
```

FAIL CLOSED If signature, amount, currency or payment-intent reconciliation fails, the payment must not become authoritative as confirmed.

33 PAYMENT STATUS

```
CREATED -> ORDER_CREATED -> PENDING -> AUTHORIZED -> CAPTURED -> CONFIRMED

PENDING -> FAILED
AUTHORIZED -> FAILED
```

Use the existing payment vocabulary if an equivalent model already exists. Gateway status and StarVnt business status are related but not automatically identical.

34 PAYMENT CONFIRMATION

```
GATEWAY RESULT -> SIGNATURE VERIFIED -> WEBHOOK VERIFIED
-> AMOUNT VERIFIED -> CURRENCY VERIFIED -> PAYMENT INTENT MATCHED
-> TRANSACTION UPDATED -> PAYMENT_CONFIRMED
```

35 WEBHOOK ARCHITECTURE

```
RAZORPAY -> WEBHOOK -> RAW BODY -> SIGNATURE VERIFICATION
-> EVENT IDENTIFICATION -> IDEMPOTENCY CHECK -> PAYMENT TRANSACTION UPDATE
-> DOMAIN EVENT -> OUTBOX -> WORKER -> CONSUMERS
```

The flow must remain correct if the customer closes the browser, loses network, refreshes, leaves checkout or experiences frontend failure. Browser callback must not be the only source of payment truth.

36 PAYMENT EVENTS

```
PAYMENT_CREATED
PAYMENT_AUTHORIZED
PAYMENT_CAPTURED
PAYMENT_FAILED
PAYMENT_CONFIRMED
```

Reuse the central event infrastructure. Do not build Payment Event Bus V2 or another isolated event system.

37 PAYMENT IDEMPOTENCY

```
CREATE ORDER x N -> NO UNINTENDED DUPLICATE COMMERCIAL ORDER
WEBHOOK x100 -> ONE EFFECTIVE PAYMENT STATE
        -> ONE EFFECTIVE BUSINESS EVENT
        -> ONE EFFECTIVE DOWNSTREAM ACTION
```

```
CONFIRM x100 -> ONE EFFECTIVE CONFIRMATION
```

38 PAYMENT -> BOOKING

```
PAYMENT_CONFIRMED -> BOOKING PAYMENT DEPENDENCY SATISFIED -> BOOKING CONFIRMATION
```

Payment must not rewrite arbitrary Booking internals. Use an explicit Booking command/event contract. Booking remains the authority for Booking lifecycle.

39 PAYMENT -> EVENT CORE

```
EVENT -> REQUIREMENT -> BOOKING -> PAYMENT_REQUIRED -> PAYMENT_PENDING

PAYMENT_CONFIRMED -> BOOKING PAYMENT DEPENDENCY = READY
        -> REQUIREMENT READY
        -> EVENT READINESS RECALCULATED
```

BOUNDARY Event Core consumes the dependency/readiness effect; it does not become the Payment ledger.

40 PAYMENT -> READINESS

```
REQUIREMENT -> BOOKING -> PAYMENT DEPENDENCY -> PAYMENT CONFIRMED
-> REQUIREMENT READY -> EVENT READINESS RECALCULATED
```

The system should be able to explain the readiness change in business language, e.g. "Catering is now ready because the approved booking advance has been successfully verified."

41 PAYMENT -> AUTOMATION

```
PAYMENT_CONFIRMED -> AUTOMATION -> RULE -> ACTION
```

Possible actions include activity updates, customer/vendor notification, booking progression or the next operational step. Reuse the central Automation Engine.

42 PAYMENT -> AURA+

```
PAYMENT STATUS -> AUTHORIZED CONTEXT -> AURA+ -> ANALYSIS -> INSIGHT / RECOMMENDATION
```

Aura+ may analyze pending, confirmed or failed payment context, commercial risk and overdue payment. Aura+ remains read-only with respect to authoritative Payment/Booking/Event state unless an authorized handoff is explicitly executed by the correct domain.

```
PROHIBITED  AURA+ -> PAYMENT DATABASE -> DIRECT MUTATION
```

43 PAYMENT -> CUSTOMER EXPERIENCE

Customer status	Meaning
Payment Required	Customer action is needed
Payment Processing	Verification is underway
Payment Successful	Verified payment is authoritative
Payment Failed	Payment did not complete
Payment Verification Pending	Payment result requires server reconciliation

Do not expose webhook internals, worker state, event-bus terminology, gateway secrets, internal ledger architecture or infrastructure details.

44 AUDIT - COMMERCIAL FLOW

```
Actor | Organization | Event | Booking | Payment Intent | Transaction
Action = PAYMENT_CONFIRMED
Previous State | New State | Source = RAZORPAY_WEBHOOK | Timestamp
```

Reuse the central Audit system. Do not create a Payment-specific audit database.

45 PAYMENT SECURITY

```
CLIENT -> PUBLIC API -> AUTHORIZED SERVER -> PAYMENT DOMAIN -> RAZORPAY
```

```

```

- Never expose Razorpay secret keys or webhook secrets to the client.
- Never trust client-supplied payable amount as authoritative.
- Never treat frontend success alone as payment confirmation.
- Enforce authorization and tenant context server-side.

46 FUTURE PAYMENT INFRASTRUCTURE

Create clean extension boundaries for Refund, Settlement, Reconciliation, Installments, Partial Payment, Multi-Vendor Allocation, Platform Fee, Tax Allocation, Cancellation Refund, Escrow, Milestone Payment, Payout and Dispute - but do not build fake or half-complete versions on Day 7.

```
PAYMENT
 |-- REFUND
 |-- SETTLEMENT
 |-- RECONCILIATION
 |-- INSTALLMENT
 |-- PARTIAL PAYMENT
 |-- ESCROW
 |-- MILESTONE
 |-- PAYOUT
 |-- PLATFORM FEE
 |-- TAX
 |-- MULTI-VENDOR ALLOCATION
 `-- DISPUTE
```

RULE Architecture-ready does not mean UI-ready.

47 FUTURE ESCROW / REFUND / SETTLEMENT COMPATIBILITY

```
CUSTOMER MONEY -> PAYMENT -> BOOKING OBLIGATION -> ESCROW / HOLD
-> MILESTONE -> RELEASE -> SETTLEMENT

PAYMENT -> REFUND
PAYMENT -> SETTLEMENT
INTERNAL TRANSACTIONS <-> GATEWAY RECORDS -> RECONCILIATION
```

48 QR / ENTRY + REALTIME COMPATIBILITY

The Event Core should expose enough lifecycle/access context for existing QR/Entry and Realtime capabilities to consume. Do not build those systems anew.

```
EVENT -> EVENT ACCESS CONTEXT -> QR / ENTRY -> VALIDATION -> ENTRY RESULT
EVENT CORE -> CANONICAL EVENT -> REALTIME HUB -> CLIENTS
```

49 FUTURE DIGITAL TWIN / EVENT MEMORY / HEALTH

The future Digital Twin can derive Event, people, guests, vendors, requirements, resources, commitments, dependencies, costs, payments, timeline, state, history and risks through domain contracts. Event Memory can later derive planned vs actual, budget vs actual, vendor performance, delays, issues, decisions, payment history and booking history. Day 6/7 should preserve the necessary history without building duplicate representations.

50 COMPLETE DAY 6 + DAY 7 COMMERCIAL FLOW

```
CUSTOMER -> LEAD -> QUALIFICATION -> EVENT -> EVENT CONTEXT
-> APPLICABILITY -> REQUIREMENT -> RESOURCE / QUANTITY -> VENDOR MATCHING
-> QUOTE -> CUSTOMER APPROVAL -> BOOKING -> PAYMENT INTENT
-> RAZORPAY ORDER -> CHECKOUT -> SERVER VERIFICATION -> WEBHOOK
-> PAYMENT TRANSACTION -> PAYMENT_CONFIRMED -> OUTBOX -> WORKER
-> BOOKING DEPENDENCY SATISFIED -> BOOKING CONFIRMED
-> EVENT READINESS -> AUTOMATION -> CUSTOMER -> AURA+
```

51 FAILURE TESTING - EVENT CORE

Test	Scenario	Expected
01	Unauthorized user -> CONFIRM EVENT	Denied
02	ORG-B user -> ORG-A Event	Denied; no leak
03	DRAFT -> COMPLETED	Denied; no state change
04	CONFIRM x100	One effective transition
05	Event -> Outbox -> Worker failure	Retry + recovery
06	Aura+ recommendation -> handoff	Controlled Core execution

52 FAILURE TESTING - PAYMENT

Test	Scenario	Expected
01	Unauthorized -> CREATE PAYMENT	Denied
02	ORG-B -> ORG-A payment	Denied
03	Client modifies amount	Rejected
04	Invalid checkout signature	Payment not confirmed
05	Invalid webhook signature	No payment state change
06	Webhook x100	One effective payment
07	Browser disconnects	Webhook path still reconciles
08	Worker failure	Retry + recovery
09	Duplicate confirmation	One effective confirmation

53 REAL RAZORPAY TEST-MODE PROOF

```
REAL STARVNT BOOKING
 -> REAL PAYMENT INTENT
 -> REAL RAZORPAY TEST ORDER
 -> REAL CHECKOUT
 -> REAL TEST PAYMENT
 -> REAL SERVER VERIFICATION
 -> REAL WEBHOOK
 -> REAL PAYMENT TRANSACTION
 -> REAL PAYMENT_CONFIRMED
 -> REAL OUTBOX
 -> REAL WORKER
 -> REAL BOOKING UPDATE
 -> REAL EVENT READINESS UPDATE
 -> REAL AUTOMATION
 -> REAL CUSTOMER STATUS
```

54 REAL E2E PROOF - DAY 6 + DAY 7

```
LOGIN -> ORGANIZATION -> ROLE -> CREATE EVENT -> DRAFT -> REQUIREMENTS / CHECKLIST
-> READINESS -> MANIFESTING -> VENDOR / MATCHING -> QUOTE -> CUSTOMER APPROVES
-> BOOKING -> PAYMENT INTENT -> RAZORPAY TEST ORDER -> CHECKOUT
-> SERVER VERIFICATION -> WEBHOOK VERIFICATION -> PAYMENT TRANSACTION
-> PAYMENT_CONFIRMED -> OUTBOX -> WORKER -> BOOKING PAYMENT DEPENDENCY READY
-> BOOKING CONFIRMED -> EVENT READINESS RECALCULATED -> AUTOMATION -> AUDIT
-> CUSTOMER STATUS -> AURA+ READ-ONLY CONTEXT
```

EVIDENCE STANDARD Every critical step must be traceable to real API, database, event, outbox, worker and audit evidence.

55 DAY 6 ACCEPTANCE GATES

Area	Required
Identity Integration	YES
Organization	YES
Membership	YES
RBAC	YES
Event Creation	YES
Event State Machine	YES
Event Permissioning	YES
Tenant Isolation	YES
Checklist Integration	YES
Applicability Compatibility	YES
Requirement Integration	YES
Readiness Validation	YES
Domain Events	YES

Area	Required
Event Contract	YES
Transactional Outbox	YES
Worker Integration	YES
Automation Integration	YES
Audit	YES
Idempotency	YES
Invalid Transition Protection	YES
Cross-Tenant Protection	YES
Aura+ Boundary	YES
Vendor/Booking Boundary	YES
Payment Boundary	YES
QR/Entry Boundary	YES
Realtime Boundary	YES
E2E Event Lifecycle	YES
Failure Recovery	YES

56 DAY 7 ACCEPTANCE GATES

Area	Required
Existing Revenue Integration	YES
Requirement Integration	YES

Area	Required
Vendor Integration	YES
Matching Integration	YES
Quote Integration	YES
Customer Approval	YES
Booking Integration	YES
Payment Intent	YES
Payment Transaction	YES
Razorpay Test Mode	YES
Razorpay Order Creation	YES
Checkout	YES
Server-side Verification	YES
Checkout Signature Verification	YES
Webhook Signature Verification	YES
Raw Body Verification	YES
Amount Reconciliation	YES
Currency Reconciliation	YES
Payment Status Lifecycle	YES
Payment Events	YES
Idempotency	YES
Transactional Outbox	YES
Worker	YES

Area	Required
Booking Confirmation	YES
Event Dependency Update	YES
Event Readiness	YES
Automation Integration	YES
Audit	YES
RBAC	YES
Tenant Isolation	YES
Aura+ Read-only Contract	YES
Duplicate Webhook Protection	YES
Failure Recovery	YES
Real Razorpay Test Proof	YES
Full E2E Commercial Flow	YES

57 WHAT MUST NOT BE BUILT

- A second Event, Payment, Booking, Vendor, Automation, Audit, Realtime or Event representation system.
- A duplicate financial ledger or shadow source of truth.
- Full Settlement, Reconciliation, Refund, Escrow, Installment, Payout, Tax Allocation or Dispute systems merely for Day 7 optics.
- Fake payment success, fake settlement/refund dashboards, fake reconciliation or fake escrow.
- Client-controlled payment amounts or client-side payment confirmation.
- Aura+ direct mutation of authoritative financial, booking or Event state.
- Unnecessary UI, infrastructure or speculative payment AI.

58 SECURITY ABSOLUTES

NEVER TRUST THE FRONTEND FOR:

```
authorization | tenant identity | payment amount | payment confirmation
booking confirmation | Event state transition

SERVER MUST CONTROL:
authorization | tenant context | payable amount | payment verification
state transition | booking command | readiness effect
```

59 DATA AUTHORITY PRINCIPLE

```
DOMAIN TRUTH -> AUTHORITATIVE DOMAIN STATE
BUSINESS EVENT -> SIGNAL THAT SOMETHING HAPPENED

Payment -> Payment truth
Booking -> Booking truth
Vendor -> Vendor truth
Requirement -> Requirement truth
Event Core -> Event lifecycle truth
Aura+ -> Intelligence truth
```

60 CUSTOMER EXPERIENCE PRINCIPLE

The backend may be complex, but the customer experience must stay simple. The customer should understand where they are, what is ready, what is pending, who is handling it, what needs approval, what must be paid and what happens next.

HIDE Do not expose event buses, workers, outboxes, microservices, webhook internals, database schemas or gateway architecture to customers.

61 ENGINEERING EXECUTION RULE

```
REVIEW -> TRACE -> UNDERSTAND -> IDENTIFY DOMAIN OWNER
-> CHECK EXISTING CAPABILITY -> REUSE -> DEFINE BOUNDARY
-> EXTEND -> INTEGRATE -> TEST -> BREAK -> RECOVER -> PROVE
```

DISCIPLINE Reuse > Rebuild | Integration > Duplication | Server Authority > Frontend Trust Idempotency > Optimistic Assumptions | Real Transaction > Mock Demo | Working Proof > Documentation

62 MASTER DEFINITION OF DONE

This mission is NOT DONE because the Event API works, Razorpay Checkout opens, payment status changes to success, or the architecture document is complete.

It is DONE when StarVnt can authenticate and authorize a user, create and operate an Event through its controlled lifecycle, derive real requirements and readiness, connect existing Vendor/Matching/Quote/Booking capabilities, obtain customer approval, create a server-controlled Payment Intent, create a real Razorpay Test Mode order, execute checkout, independently verify the payment, verify the webhook correctly, reconcile amount and currency, record the authoritative transaction, emit business events, survive duplicate delivery and worker failure, satisfy the Booking payment dependency, update Event readiness, trigger Automation, record Audit and expose reliable commercial context to the customer and Aura+ - all without violating domain ownership or creating duplicate systems.

63 FUTURE PAYMENT EXTENSION MODEL

```
PAYMENT
 |-- REFUND
 |-- SETTLEMENT
 |-- RECONCILIATION
 |-- INSTALLMENT
 |-- PARTIAL PAYMENT
 |-- ESCROW
 |-- MILESTONE
 |-- PAYOUT
 |-- PLATFORM FEE
 |-- TAX
 |-- MULTI-VENDOR ALLOCATION
 `-- DISPUTE
```

These remain future capabilities unless explicitly implemented later. Architecture-ready means the current model can evolve safely; it does not mean the feature is already live.

64 DAY 6 -> DAY 7 -> DAY 8+

```
DAY 6
EVENT CORE AUTHORITY
 |-- Lifecycle
 |-- Requirements
 |-- Readiness
 |-- Events
 |-- Outbox
 |-- Worker
 |-- Automation
 `-- Audit
       |
       v
TRUSTWORTHY EVENT CORE
       |
```

```
      v
DAY 7
EVENT COMMERCE OPERATING FLOW
 |-- Vendor / Matching
 |-- Quote / Approval
 |-- Booking
 |-- Payment Intent
 |-- Razorpay
 |-- Verification / Webhook
 |-- Payment Transaction
 |-- Booking Confirmation
 |-- Event Readiness
 `-- Automation / Aura+ Context
      |
      v
REAL COMMERCIAL EVENT EXECUTION
      |
      v
DAY 8+
EVENT OPERATING SYSTEM
 |-- Optimization
 |-- Event Health
 |-- Intelligence
 |-- Execution
 |-- Prediction
 |-- Escrow / Settlement
 |-- Reconciliation / Payout
 `-- Event Memory
```

65 MASTER ARCHITECTURAL NORTH STAR





66 FINAL COMMAND TO KESHAB

Keshab, Day 6 and Day 7 are one continuous Core execution mission.

Day 6 is responsible for making the Event Core trustworthy. Day 7 is responsible for proving that the trustworthy Event Core can support a real commercial transaction.

Review the existing StarVnt implementation first across Identity, Organization, Membership, RBAC, Customer/Core, Revenue, Requirement, Vendor, Matching, Quote, Booking, Automation, Outbox, Worker, Audit and Reliability. Do not assume something is missing because it is not visible in one module.

Reuse existing capabilities wherever they already exist. Do not create duplicate systems. Establish the Event Core lifecycle DRAFT -> MANIFESTING -> CONFIRMED -> ONGOING -> COMPLETED with authorization, validation, tenant isolation, idempotency, audit and business-event/outbox/worker integration.

Then connect that operational authority to the commercial flow Requirement -> Vendor Match -> Quote -> Customer Approval -> Booking -> Payment Intent -> Razorpay Order -> Checkout -> Server Verification -> Webhook -> Payment Transaction -> Payment Confirmed -> Booking Dependency -> Event Readiness -> Automation -> Customer/Aura+.

Use real Razorpay Test Mode. Do not trust frontend payment success or client-provided amounts. Verify payment server-side, verify webhook authenticity correctly, reconcile amount and currency, process duplicates idempotently, and reuse the existing Outbox and Worker infrastructure.

Connect verified payment state to Booking and Event Core through explicit contracts. Update Event readiness based on actual dependencies. Provide Aura+ only with authorized read-only context. Never permit Aura+ to directly mutate authoritative Payment, Booking or Event state.

Keep Refund, Settlement, Reconciliation, Installments, Partial Payment, Multi-Vendor Allocation, Platform Fees, Tax Allocation, Cancellation Refund, Escrow, Milestone Payments, Payout and Dispute architecture-ready, but do not build fake or half-complete versions to claim future readiness.

Finally, prove the complete system with real API, database, Event lifecycle, Booking, Payment Intent, Razorpay Test Mode, Checkout, server verification, webhook, Payment Transaction, Outbox, Worker, Booking update, Event readiness update, Automation, Audit, customer status and Aura+ read-only context. Then deliberately attack the system and prove that it fails safely and recovers correctly.

FINAL STANDARD Working proof > documentation | Integration > duplication | Server authority > frontend trust | Idempotency > optimistic assumptions | Real transaction flow > isolated gateway demo

67 FINAL ARCHITECTURAL STATEMENT

DAY 6 Makes StarVnt's Event Core trustworthy.

DAY 7 Proves that StarVnt can execute a trustworthy commercial transaction on top of that Core.

EVENT AUTHORITY -> COMMERCIAL AUTHORITY -> VERIFIED TRANSACTION -> OPERATIONAL READINESS

ONE STARVNT One Event Core. One connected commerce operating model. One authoritative operating model.

APPENDIX A EXECUTION CHECKLIST - KESHAB HANDOFF

Use this page as the final pre-demo gate. Every item should be backed by working evidence, existing implementation reference, or an explicitly documented dependency/blocker.

Checkpoint	Status	Checkpoint	Status
1. Day 6 Event lifecycle works end-to-end	PASS / FAIL	2. Event transitions are permission-controlled	PASS / FAIL
3. Cross-tenant Event access is denied	PASS / FAIL	4. Invalid transitions create no false events	PASS / FAIL
5. Checklist/Requirement feeds actual readiness	PASS / FAIL	6. Canonical Event events use existing infrastructure	PASS / FAIL
7. Outbox is transactional with state changes	PASS / FAIL	8. Worker retry/recovery works	PASS / FAIL
9. Audit captures lifecycle changes	PASS / FAIL	10. Duplicate commands are idempotent	PASS / FAIL
11. Aura+ cannot directly mutate authoritative state	PASS / FAIL	12. Vendor/Booking/Payment boundaries are enforced	PASS / FAIL
13. Day 7 integrates existing Revenue/Matching/Quote/Booking	PASS / FAIL	14. Payment Intent is server-controlled	PASS / FAIL
15. Razorpay Test Order is real	PASS / FAIL	16. Checkout is real Test Mode	PASS / FAIL
17. Checkout signature is verified server-side	PASS / FAIL	18. Webhook signature uses correct raw-body verification	PASS / FAIL
19. Amount is reconciled against authoritative StarVnt context	PASS / FAIL	20. Currency is reconciled	PASS / FAIL
21. Payment Transaction records the gateway outcome	PASS / FAIL	22. Duplicate webhooks are idempotent	PASS / FAIL
23. Payment confirmation triggers the right domain event	PASS / FAIL	24. Outbox/Worker carries payment events downstream	PASS / FAIL
25. Booking payment dependency is satisfied after verified payment	PASS / FAIL	26. Event readiness is recalculated	PASS / FAIL
27. Automation runs through the central engine	PASS / FAIL	28. Customer receives a clean payment status	PASS / FAIL
29. Aura+ receives read-only payment context	PASS / FAIL	30. Failure/Recovery tests are	PASS / FAIL

Checkpoint	Status	Checkpoint	Status
		demonstrated	
31. Full Day 6 + Day 7 E2E is demonstrated	PASS / FAIL	32. No duplicate system has been introduced	PASS / FAIL
33. Future payment features remain architecture-ready, not fake-built	PASS / FAIL	34.	

EVIDENCE PACK + FINAL SIGN-OFF

The completion package should contain working evidence rather than claims. The review package should make it possible to independently trace the commercial flow from Event authority through verified payment and downstream readiness.

Evidence artifact	Minimum proof
API evidence	Requests/responses for Event, Booking, Payment Intent, verification and state transitions
Database evidence	Authoritative Event, Booking, Payment Intent and Transaction state after the run
Gateway evidence	Razorpay Test Mode order/payment identifiers and verified webhook path
Reliability evidence	Outbox record, Worker processing, retry behaviour and duplicate protection
Audit evidence	Lifecycle + payment audit trail with source, actor and timestamps
Failure evidence	Unauthorized, cross-tenant, signature, amount, duplicate and Worker-failure tests
E2E evidence	One end-to-end recording or trace showing the complete Day 6 + Day 7 flow

FINAL REVIEW No critical gate is considered complete until the implementation is traceable to working evidence in the repository and runtime.